

Simulación de Entrevista Técnica — Docker

Guía de estudio en formato entrevista · Docker, multi-stage builds, BuildKit y Docker Compose en el sistema POS

Guía de estudio en formato entrevista sobre **Docker** y **Docker Compose** en este proyecto POS (6 servicios Java + 3 frontends Angular + infraestructura). Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo** y **follow-ups**.

Formato sugerido: 45–55 min. Bloques: fundamentos (10'), imágenes y Dockerfile (15'), Compose (10'), optimización de builds del proyecto (10'), operación (10').

0. Warm-up — Docker en el proyecto

P ¿Cómo se conteneriza y orquesta este sistema en local?

R Todo corre con **Docker Compose**: infraestructura (Kafka, MongoDB, PostgreSQL, Redis, Eureka), los 6 servicios Java y los 3 frontends Angular. Los servicios Java usan Dockerfiles **multi-stage** con caché de Gradle compartida (BuildKit); los frontends compilan con Node y sirven el estático con **Nginx**. El tooling pesado (Jenkins, SonarQube) está detrás de **perfiles** de Compose y no arranca por defecto. Hay scripts de arranque por fases (`start-stack.sh`) para no saturar la máquina.

1. Fundamentos

P1 ¿Diferencia entre contenedor y máquina virtual?

R Una **VM** virtualiza hardware y corre un **SO invitado completo** sobre un hypervisor: aislamiento fuerte pero pesado (GBs, arranque lento). Un **contenedor** comparte el **kernel del host** y aísla procesos con **namespaces** (PID, net, mount) y **cgroups** (límites de CPU/memoria): mucho más ligero (MBs, arranque en ms) pero con aislamiento a nivel de proceso. Por eso Docker es ideal para microservicios: densidad y velocidad.

P2 ¿Imagen vs contenedor?

R Una **imagen** es una plantilla inmutable de solo lectura, construida por capas. Un **contenedor** es una **instancia en ejecución** de una imagen, con una capa de escritura efímera encima. Analogía: imagen = clase, contenedor = objeto instanciado. Muchos contenedores pueden nacer de la misma imagen.

2. Imágenes y Dockerfile

P3 Explica los multi-stage builds y por qué se usan aquí.

R Un **multi-stage build** usa varias etapas. `FROM` : una etapa *builder* con el toolchain pesado (JDK+Gradle o Node) compila el artefacto, y una etapa *runtime* ligera (JRE o Nginx) copia solo el resultado final. Así la imagen final **no arrastra** compiladores ni dependencias de build → imágenes pequeñas y con menor superficie de ataque.

```
# etapa build
FROM gradle:8.9-jdk21 AS build
WORKDIR /app
COPY . .
RUN gradle :stock-service:bootJar

# etapa runtime (slim)
FROM eclipse-temurin:21-jre
COPY --from=build /app/stock-service/build/libs/*.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

Los frontends hacen lo mismo: etapa Node hace `ng build`, etapa Nginx sirve `dist/`.

P4 ¿Cómo funciona el layer caching y cómo lo aprovechas?

R Cada instrucción del Dockerfile crea una **capa** cacheada. Docker reutiliza una capa si su instrucción y su contexto no cambiaron. La regla clave: poner lo que **cambia poco arriba** y lo que cambia mucho abajo. Por eso se copian primero los ficheros de dependencias (`build.gradle` / `package.json`) y se descargan dependencias, y **luego** el código fuente: así un cambio de código no invalida la capa de dependencias.

FOLLOW-UP ¿Cómo reduces el tamaño de imagen? — Imagen base slim/alpine, multi-stage, `.dockerignore`, combinar `RUN` y limpiar cachés de apt/npm en la misma capa, y no instalar dependencias de dev.

P5 ¿Por qué el frontend "no muestra los cambios" hasta reconstruir?

R Porque el build de Angular queda **congelado dentro de la imagen**: Nginx sirve el `dist/` horneado en tiempo de build. Editar el código fuente no cambia lo servido hasta **reconstruir la imagen y recrear el contenedor**: `docker compose up -d --build --force-recreate <servicio>`. Angular pone hash en los bundles (`main-XXXX.js`), lo que invalida caché del navegador tras el redeploy.

3. Docker Compose

P6 ¿Qué aporta Docker Compose y qué son los perfiles?

R Compose declara el stack multi-contenedor en un YAML (servicios, redes, volúmenes, dependencias) y lo levanta con un comando. Los **perfiles** permiten agrupar servicios opcionales que no arrancan por defecto: aquí `tooling / sonar / ci` (Jenkins, SonarQube) quedan fuera del arranque normal, dejando solo los ~13 contenedores esenciales. Se activan con `docker compose --profile tooling up -d`.

P7 ¿Cómo manejas el orden de arranque y las dependencias?

R Con `depends_on` combinado con **healthchecks** (`condition: service_healthy`): un servicio Java no arranca hasta que Kafka/Mongo están *healthy*, no solo *started*. Aun así, la app debe tolerar reintentos de conexión. El proyecto además usa arranque **por fases** (infra → backend → frontend) para no saturar CPU con 6 JVMs a la vez.

FOLLOW-UP ¿ `depends_on` espera a que la app esté lista? — Sin healthcheck solo espera a que el contenedor *arranque*, no a que la app *escuche*. Por eso se define `healthcheck` (p. ej. `/actuator/health`) y se usa `condition: service_healthy`.

4. Optimización de builds (proyecto)

P8 ¿Qué es BuildKit y cómo lo usa el proyecto?

R BuildKit es el motor de build moderno de Docker: builds en paralelo, mejor caché y **cache mounts**. El proyecto comparte una **caché de Gradle** (`/root/.gradle`) entre los 6 servicios y una **caché de npm** (`/root/.npm`) entre frontends, de modo que las dependencias se descargan **una sola vez**. Requiere `DOCKER_BUILDKIT=1` (los scripts de arranque lo activan).

P9 ¿Qué hace `build-images-fast.sh` y por qué es más rápido?

R En vez de lanzar un Gradle **dentro de cada contenedor** (lento y sin caché compartida), compila **todos los JAR en un solo build de Gradle en el host** (compartiendo la build cache) y los empaqueta en imágenes **slim solo-JRE** (`Dockerfile.prebuilt`). Esto baja un rebuild completo de ~15-25 min a ~2.5 min en un host de 4 CPU.

5. Redes, volúmenes y operación

P10 ¿Cómo se comunican los contenedores entre sí?

R Compose crea una **red bridge** por proyecto y da **DNS interno**: cada servicio es alcanzable por su **nombre** (p. ej. `kafka:9092` , `mongodb:27017`). Solo los puertos **publicados** (`ports:`) quedan expuestos al host. El aislamiento de red evita exponer todo hacia fuera.

P11 ¿Cómo persisten datos MongoDB/PostgreSQL entre reinicios?

R Con **volúmenes** nombrados montados en los directorios de datos. El sistema de ficheros del contenedor es **efímero** (se pierde al recrear); un volumen vive fuera del ciclo de vida del contenedor, así que los datos sobreviven a `up/down` . `docker compose down -v` **borra** también los volúmenes (destructivo).

6. Diseño abierto / escenarios

P12 Buenas prácticas de seguridad en imágenes de contenedor.

R Correr como **usuario no-root** (`USER`), imágenes base **mínimas** (distroless/slim) para reducir superficie, **fijar versiones** (no `latest`), escanear vulnerabilidades (Trivy/Grype), no meter **secretos** en capas (usar variables/secret mounts), y filesystem de solo lectura cuando se pueda. El `JWT_SECRET` y credenciales van por variables de entorno / secretos, nunca horneados en la imagen.

P13 El arranque satura la máquina. ¿Qué ajustas?

R El arranque de Spring Boot es **CPU-bound** (JIT, class scanning). El proyecto asigna **2 CPU / 768M** por JVM; bajarlo a 0.5 CPU casi duplica el tiempo. Se combina con arranque **por fases** (esperar health checks entre capas) y `-Djava.security.egd=file:/dev/./urandom` para no bloquear por entropía. Perfiles de Compose evitan levantar tooling innecesario.

Apéndice — Chuleta rápida

Tema	Punto clave
Contenedor vs VM	Comparte kernel (namespaces + cgroups); ligero vs SO invitado completo
Imagen vs contenedor	Plantilla inmutable por capas vs instancia en ejecución
Multi-stage	Builder pesado → runtime slim (JRE/Nginx); imágenes pequeñas y seguras
Layer cache	Dependencias arriba, código abajo; no invalidar capas caras
Frontend	Build horneado en la imagen; rebuild + force-recreate para desplegar
Compose	Stack en YAML; perfiles dejan tooling fuera del arranque por defecto
Orden	<code>depends_on</code> + <code>healthcheck (service_healthy)</code> + fases
BuildKit	Cache mounts de Gradle/npm compartidas; <code>DOCKER_BUILDKIT=1</code>
Red/volúmenes	DNS interno por nombre; volúmenes para persistir datos (<code>down -v borra</code>)
Seguridad	No-root, base mínima, versiones fijas, escaneo, secretos fuera de la imagen